

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Зотеев Максим

БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить брокер сообщений (RabbitMQ) к микросервисному бэкенд-приложению, реализованному в ЛР2. Реализовать асинхронное межсервисное взаимодействие через очереди сообщений: одни сервисы должны публиковать события о значимых бизнес-действиях, другие - подписываться и реагировать на них, не зная о существовании источника событий напрямую

Ход работы

1. Зачем понадобился брокер сообщений

В микросервисной архитектуре ЛР2 межсервисное взаимодействие сделано синхронно по HTTP. Такой подход подходит для запросов «получить контекст», «проверить участника сделки», где вызывающему нужен немедленный ответ. Но он плохо подходит для нотификаций «случилось событие X, кому интересно - обработайте». Если rental-service после успешного создания сделки сам ходил бы в messaging-service «допиши системное сообщение в чат», то падение messaging-service ломало бы создание сделки, а добавление новых подписчиков (например, будущего notification-service для писем) потребовало бы правок в rental-service. Брокер сообщений снимает эту связность: rental-service публикует факт «сделка создана», а кто и как на него реагирует - забота подписчиков

2. Выбор технологии и подключение

В качестве брокера выбран RabbitMQ 3.13 с включённым management plugin. Он запускается отдельным контейнером в общем docker-compose.yml, healthcheck опирается на rabbitmq-diagnostics ping, для разработки доступна web-консоль на <http://localhost:15672>. Все сервисы, использующие брокер (identity, rental, messaging), подключаются через библиотеку amqp-lib. URL брокера передаётся переменной окружения RABBITMQ_URL. Подключение выполняется на старте с ретраями (10 попыток с интервалом 2 секунды), чтобы переждать поднятие RabbitMQ при первом запуске docker compose

3. Топология обмена

В качестве обмена используется один exchange rental.events типа topic, объявленный durable (переживает рестарт брокера). Тип topic выбран, чтобы подписчики могли фильтровать события по шаблону routing key. Producers - rental-service (для событий о сделках) и identity-service (для событий о регистрациях). Consumer - messaging-service: создаёт durable-очередь messaging.rental-events с binding key rental.*, что позволит подписаться сразу на все события из домена сделок без перенастройки при добавлении новых типов событий

4. Перечень событий и их payload

В системе определены четыре события. user.registered публикуется identity-service после успешной регистрации, содержит user_id, email, role и occurred_at. rental.created публикуется rental-service после успешной фиксации сделки и переключения

доступности объекта, содержит rental_id, property_id, property_title, tenant_id, owner_id, occurred_at. rental.completed и rental.cancelled публикуются rental-service после соответствующих переходов статуса и содержат rental_id, property_id, tenant_id, owner_id, occurred_at. Все схемы зафиксированы в файле events-schema.json в формате JSON Schema

5. Что делает consumer

Messaging-service слушает очередь messaging.rental-events. На каждое событие из rental.* он создаёт в своей БД строку в таблице messages с kind='system' и sender_id=null. Текст системного сообщения формируется из шаблона по routing key: для rental.created - «Сделка по объекту "..." создана. Желаем удачной аренды!», для rental.completed - «Сделка успешно завершена.», для rental.cancelled — «Сделка отменена.». В результате участники сделки видят прогресс прямо в общем чате сделки без отдельных нотификаций, а в API GET /rentals/:id/messages эти сообщения появляются автоматически и отдаются вместе с обычными пользовательскими

6. Гарантии доставки и обработка ошибок

Применены типовые настройки надёжности AMQP. Exchange и очередь объявлены durable, сообщения публикуются с persistent=true - после рестарта брокера ничего не теряется. Consumer обрабатывает сообщения в режиме ручного подтверждения: при успехе вызывает ack, при необработанной ошибке - nack(requeue=false), чтобы битое сообщение не закидывалось. Publisher работает в режиме fire-and-forget: если канал к брокеру по какой-то причине ещё не готов, событие логируется и пропускается - основной HTTP-поток создания сделки не должен падать из-за временной недоступности брокера

7. Проверка работы

Сквозной сценарий выполнен через Postman-коллекцию ЛР2 (тесты 48 и 51): сразу после POST /properties/:id/rentals и POST /rentals/:id/complete вызывается GET /rentals/:id/messages с небольшой паузой и проверяется, что в чате появилось системное сообщение нужного вида (kind=system, sender=null, текст соответствует шаблону). Прогон автотестов через newman подтверждает работоспособность шины событий. Дополнительно по http://localhost:15672 (rental/rental) в management-консоли видно, что счётчики publish и deliver_get на exchange rental.events растут синхронно с действиями в API

Заключение

В рамках Д35 в микросервисное приложение из ЛР2 интегрирован брокер сообщений RabbitMQ. Реализовано асинхронное межсервисное взаимодействие через единый topic exchange rental.events: rental-service публикует события жизненного цикла сделки, identity-service - события о регистрации пользователей, messaging-service подписывается на rental.* и автоматически дописывает системные сообщения в чаты соответствующих сделок. Применены настройки надёжной доставки (durable exchange и queue, persistent messages, manual ack/nack без requeue) и устойчивости к старту в

docker compose (ретраи подключения). Сервисы остаются развязанными: добавление нового подписчика (например, будущего notification-service для писем) не требует изменений ни в одном из существующих сервисов